

Zmienne

```
In [1]: 1 # To jest komentarz
2 """
3 To jest komentarz.
4 Może zawierać wiele linii.
5 Stosowany w dokumentacji klas i funkcji.
6 """
7
8 # Wyświetlanie tekstu
9 print("Hello world")
10
```

Hello world

```
In [2]: 1 # Tworzenie zmiennych
2 # Liczba całkowita
3 zmienna = 2
4
5 # Liczba rzeczywista
6 rzeczywista = 4.5
7 rzeczywista = float(34) # konwersja danych
8 rzeczywista = 34.0
9
10 # zmienna tekstowa string
11 zmiennaTekstowa1 = "jakis tekst"
12 zmiennaTekstowa2 = "to jest nowa zmienna"
13
14 # zmienna typu bool
15 is_active = True
16
17 # Łączenie i wyświetlanie tekstu
18 print(zmiennaTekstowa1 + " " + zmiennaTekstowa2)
19
```

jakis tekst to jest nowa zmienna

```
In [3]: 1 # Nie można bezpośrednio łączyć zmiennnych tekstowych i liczbowych
2 print(zmienna + zmiennaTekstowa1)
3
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-3-f6dc02bee53d> in <module>
      1 # Nie można bezpośrednio łączyć zmiennnych tekstowych i liczbowych
----> 2 print(zmienna + zmiennaTekstowa1)

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

List

```
In [4]: 1 """
2 list - jest to kolekcja wielu elementów.
3 Elementy listy są:
4 - posortowane,
5 - edytowalne (dodawanie, usuwanie, zmienianie wartości),
6 - indeksowane, pierwszy element ma index 0.
7 Na liście można przechowywać zduplikowane wartości.
8 Zapisywane są w kwadratowych nawiasach.
9 """
10 # inicjalizacja listy wraz z wartościami
11 lista_punktow = ["A", "C", "D"]
12
13 # mogą przechowywać elementy różnego typu
14 lista_danych = ["A", "C", 3, "D", 3, True]
15
```

```
In [5]: 1 # Tworzenie list
2 lista = []
3 # dodawanie nowych wartości, użycie metody append
```

```
4 lista.append(23.2)
5 lista.append(12.3)
6 lista.append(4)
7
```

```
In [7]: 1 # Sprawdzenie długości
2 print(len(lista))
3
4 # Wyświetlanie kolejnych wartości
5 print("Kolejne wartosci:")
6 print(lista[0])
7 print(lista[1])
8 print(lista[2])
9
```

```
3
Kolejne wartosci:
23.2
12.3
4
```

```
In [8]: 1 # Inny sposob na wyswietlenie kolejnych elementow listy poprzez
2 # konwersje elementow na string
3 print("Kolejne elementy: " + str(lista[0]) + ", " + str(lista[1])
4       + ", " + str(lista[2]))
5
6 # Wyświetlenie elementu i jego typu
7 print("Liczba: " + str(lista[0]) + " typ " + str(type(lista[0])))
8 print("Liczba: " + str(lista[1]) + " typ " + str(type(lista[1])))
9 print("Liczba: " + str(lista[2]) + " typ " + str(type(lista[2])))
10
```

```
Kolejne elementy: 23.2, 12.3, 4
Liczba: 23.2 typ <class 'float'>
Liczba: 12.3 typ <class 'float'>
Liczba: 4 typ <class 'int'>
```

```
In [ ]: 1 """
2 Niektóre metody:
3 - clear() - wyczyszczenie listy
4 - copy() - zwraca kopie aktualnej listy
5 - count(wartosc) - zwraca liczbę występujących elementów o podanej
6               wartości
7 - index(wartosc) - zwraca index pierwszego elementu o podanej
8               wartości
9 - sort() - sortowanie listy, domyślny argument sortujący rosnąco
10          reverse=False, dla sortowania malejącego należy podać
11          reverse=True
12
13 """
14
```

Tuple

```
In [10]: 1 """
2 tuple - jest to kolekcja wielu elementów.
3 Elementy są:
4 - posortowane,
5 - nieedytowalne,
6 - indeksowane.
7 Tuple pozwala na przechowywanie zduplikowanych wartości.
8 Zapisywane są w okrągłych nawiasach.
9 """
10 lista_punktow = ("A", "C", "D", "C")
11
12 # mogą przechowywać elementy różnego typu
13 lista_danych = ("A", "C", 3, "D", 3, True)
14
```

```
In [11]: 1 # Zwraca liczbę wystąpień danego elementu
2 lista_punktow.count("C")
3
```

Out[11]: 2

```
In [12]: 1 # Zwraca index pierwszego znalezionej elementu pasującego do
2 # przekazanej wartości
3 lista_punktow.index("C")
4
```

```
Out[12]: 1
```

Set

```
In [13]: 1 """
2 set - jest to kolekcja wielu elementów.
3 Elementy są:
4 - nieposortowane,
5 - nieedytowalne (ale można dodawać i usuwać elementy),
6 - nieindexowane.
7 Zbiór nie zawiera zduplikowanych wartości.
8 Zapisywane są w sześciennych nawiasach.
9 """
10 lista_punktow = {"A", "C", "D"}
11
12 # mogą przechowywać elementy różnego typu
13 lista_danych = {"A", "C", 3, "D", True}
14
```

```
In [14]: 1 # dodanie nowego elementu
2 lista_danych.add(34)
3
```

```
In [ ]: 1 """
2 Niektóre metody:
3 - clear() - wyczyszczenie listy
4 - copy() - zwraca kopie aktualnej listy
5 - difference(set2) - zwraca różnicę elementów pomiędzy pierwszym
6                   a drugim zbiorem set2
7 - pop() - usuwa losowy element ze zbioru
8 - remove(element) - usuwa przekazany element jako argument ze zbioru
9 - update(set2) - aktualizuje zbiór poprzez dodanie wartości z
10                  drugiego zbioru set2
11 """
12
```

Dict

```
In [15]: 1 """
2 dict - jest to kolekcja wielu elementów.
3 Elementy są:
4 - nieposortowane (od wersji 3.7 wprowadzono sortowanie),
5 - edytowalne,
6 - zawierają unikalne klucze.
7
8 Jest to słownik przechowujący elementy pod postacią klucz wartość.
9 Zapisany w nawiasach sześciennych.
10 """
11 punkty = {
12     "A": 3,
13     "B": 2,
14     "C": 3,
15     "nazwa": "zbiór"
16 }
17
18 # dodanie nowego elementu do słownika
19 punkty["nowy_klucz"] = 31
20
```

```
In [ ]: 1 """
2 Niektóre metody:
3 - clear() - wyczyszczenie listy
4 - copy() - zwraca kopie aktualnej listy
5 - pop(klucz) - usuwa element o podanym kluczu
6 - keys() - zwraca listę z kluczami elementów
7 - values() - zwraca listę z wartościami elementów słownika
8 - update(dict2) - aktualizuje słownik o podane wartości. Jeśli niewystępowały to
```

```
9
10
11 """
12
```

są dodane nowe, a jeśli już były dodane klucze to wartości dla nich są aktualizowane.

Sprawdzenie typu zmiennej

```
In [16]: 1 # Sprawdzenie typu zmiennej
2 print(type(zmienna))
3 print(type(rzeczywista))
4 print(type(zmiennaTekstowa1))
5
```

```
<class 'int'>
<class 'float'>
<class 'str'>
```

Operacje na obiektach

```
In [19]: 1 """
2 Operacje matematyczne.
3 - dodawania(+),
4 - odejmowania(-),
5 - mnożenia(*),
6 - dzielenia(/),
7 - operacji modulo(%),
8 - potęgowania (**),
9 - dzielenia bez reszty (/).
10
11 """
12
13 # Podnoszenie do potęgi
14 kwadrat = 3**2
15 szescian = 6**3
16 print("kwadrat " + str(kwadrat))
17 print("sześćcian " + str(szescian))
18
19 # dzielenie bez reszty
20 wynik = 15//2
21 print("wynik dzielenia 15/2 bez reszty: " + str(wynik))
22
```

```
kwadrat 9
sześćcian 216
wynik dzielenia 15/2 bez reszty: 7
```

```
In [17]: 1 """
2 Operatory porównania.
3 - równe (==),
4 - różne (!=),
5 - większe niż (>),
6 - mniejsze niż (<),
7 - większe lub równe (>=),
8 - mniejsze lub równe (<=),
9 """
10 a = 4
11 b = 7
12 print("a == 4 " + str(a == 4)) # True
13 print("a != 4 " + str(a != 4)) # False
14 print("a == 5 " + str(a == 5)) # False
15 print("a > 1 " + str(a > 1)) # True
16 #zmiana wartości wyrażenia na przeciwną
17 print("not a == 4 " + str(not a == 4))
18
```

```
a == 4 True
a != 4 False
a == 5 False
a > 1 True
not a == 4 False
```

```
In [18]: 1 """
2 Operatory logiczne.
```

```

3 - and - iloczyn logiczny
4 - or - suma logiczna
5 - not - zaprzeczenie
6 """
7 print("a == 4 and b > 3" + str(a == 4 and b > 3)) #True
8 print("a == 2 or b > 2" + str(a == 2 or b > 2)) #True
a == 4 and b > 3 True
a == 2 or b > 2 True

```

In [19]:

```

1 # Do sprawdzenia czy podany operator znajduje się w zbiorze służy operator in
2 tablica = ["abc", "def", "ghi"]
3 print("def" in tablica) # true
4 print("test" in tablica) # false
5
True
False

```

In [20]:

```

1 # Do sprawdzenia czy zmienne wskazują na ten sam obszar w pamięci służy operator is
2 x = [5, 3, 3]
3 y = [2, 6, 3]
4
5 tablica2 = [4, x, y]
6
7 print("x == tablica2[1]" + str(x == tablica2[1])) # True
8 print("x is tablica2[1]" + str(x is tablica2[1])) # True
9 print("x is tablica2" + str(x is tablica2)) # False
10
x == tablica2[1] True
x is tablica2[1] True
x is tablica2 False

```

Formatowanie tekstu i obsługa napisów

In [21]:

```

1 # Łączenie napisów
2 tekst = "witaj" + " " + "swiecie"
3

```

In [22]:

```

1 # Operacja "mnożenia" napisów pozwala na wygenerowanie powtarzających się
2 # sekwencji danego napisu
3 witaj = "witaj " * 10
4 print(witaj)
5
witaj witaj witaj witaj witaj witaj witaj witaj witaj witaj

```

In [23]:

```

1 print("Tablice o powtarzającym się ciągu elementów")
2 print([4,5] * 4)
3
Tablice o powtarzającym się ciągu elementów
[4, 5, 4, 5, 4, 5, 4, 5]

```

In [25]:

```

1 """
2 Podstawowe specyfikatory do formatowania napisów
3 %s - napis lub każdy inny obiekt mający reprezentację w formie napisu
4 %d - liczby całkowite
5 %f - liczby rzeczywiste
6 %.<ilosc cyfr> - liczba zmiennie przecinkowa ze stałą liczbą cyfr po kropce
7 """
8
9 # Dla większej liczby wyświetlanych zmiennych, zmienne powinny znaleźć się po
10 # znaku % w nawiasie
11 print("Kolejne elementy: %f, %.1f, %d" % (lista[0], lista[1], lista[2]))
12 print("Element listy %f oraz typ %s" % (lista[0], type(lista[0])))
13 print("Element listy {:f} oraz jej typ {}".format(lista[0], type(lista[0])))
14 print()
15
16 # Dla pojedynczej zmiennej nie ma konieczności umieszczania jej w nawiasie
17 print("Element %f" % lista[0])
18 print("Element {:f}".format(lista[0]))
19 print()
20

```

```

21 # Ograniczenie wyświetlania liczb znajdujących się po przecinku przy
22 # wyświetlaniu
23 print("Element %.1f" % lista[0])
24 print("Element {:.1f}".format(lista[0]))

```

25
Kolejne elementy: 23.200000, 12.3, 4
Element listy 23.200000 oraz typ <class 'float'>
Element listy 23.200000 oraz jej typ <class 'float'>

Element 23.200000
Element 23.200000

Element 23.2
Element 23.2

In [26]:

```

1 # Sprawdzenie liczby przechowywanych znaków w zmiennej tekstowej. Podobnie jak
2 # dla tablic wykorzystywana jest
3 # funkcja len(nazwa_zmiennej)
4 tekst = "To jest przykładowy tekst."
5 print(len(tekst))
6
7 # zwrócenie pierwszego występującego indeksu danego znaku lub ciąg znaków
8 index = tekst.index("jest")
9 print("index: %d" % index)
10

```

26
index: 3

In [27]:

```

1 # Zliczenie liczby występujących znaków lub ciąg znaków
2 print("Liczba występujących ciągów znaków 'k': %s" % tekst.count("k"))
3

```

Liczba występujących ciągów znaków 'k': 2

In [28]:

```

1 # Dostęp do pojedynczego znaku
2 print(tekst[1])
3

```

o

In [29]:

```

1 # Dostęp do fragmentu tekstu znajdującego się pomiędzy wskazanymi indeksami
2 print(tekst[2:5])
3

```

je

In [30]:

```

1 # Wyświetlenie całego napisu od początku do wskazanego indeksu znaku
2 print(tekst[0:5])
3 print(tekst[:5])
4

```

To je
To je

In [31]:

```

1 # Wyświetlenie całego napisu od wskazanego znaku do końca tekstu
2 print(tekst[5:len(tekst)])
3 print(tekst[5:])
4

```

st przykładowy tekst.
st przykładowy tekst.

In [32]:

```

1 # Można również liczyć i wyświetlać znaki od końca. W tym celu podaje się
2 # ujemne wartości indeksów
3 print(tekst[-1]) # Wyświetlenie ostatniego znaku
4 print(tekst[-5:-3]) # Wyświetlenie znaków pomiędzy 5 (włącznie) pozycją
5 # od końca a 3 (włącznie)
6 print(tekst[-5:]) # Wyświetlenie znaków od 5 (włącznie) pozycji od końca
7 # do samego końca
8 print(tekst[: -5]) # Wyświetlenie wszystkich znaków od początek do 5 znaku
9 # od końca (pomijany jest 5 znak od końca)
10

```

.
ek
ekst.
To jest przykładowy t

```
In [33]: 1 # Zamiana wszystkich liter na duże lub małe
2 print(tekst.upper())
3 print(tekst.lower())
4
```

TO JEST PRZYKŁADOWY TEKST.
to jest przykładowy tekst.

```
In [34]: 1 # Sprawdzenie czy napis zaczyna sie lub konczy od podanego ciagu znakow
2 print(tekst.startswith("To "))
3 print(tekst.startswith("tekst."))
4 print(tekst.endswith("To "))
5 print(tekst.endswith("tekst."))
6
```

True
False
False
True

```
In [35]: 1 # Rozdzielanie napisu i zapisywanie go w tablicy np. w poniższym przykładzie
2 # spacja odpowiada za rozdzielenie tekstu
3 tablica_rozdzielona = tekst.split(" ")
4 print(tablica_rozdzielona)
5
```

['To', 'jest', 'przykładowy', 'tekst.']

Instrukcja warunkowa If ... Else

```
In [ ]: 1 # Dla instrukcji warunkowych należy pamiętać o wcięciach w programie
2 if warunek: # warunek z wykorzystaniem operatorów porównania i logicznych
3     jakas_akcja
4 elif warunek2:
5     jakas_inna_akcja
6 else: # wykonanie programu, gdy żaden z powyższych warunków nie został
7     # spełniony.
8     jakas_jeszcze_inna_akcja
9
```

```
In [36]: 1 # Przykład użycia
2 a = 3
3 if a == 5:
4     print("zmienna a jest równa 5")
5 elif a > 5:
6     print("zmienna a jest większa od 5")
7 else:
8     print("zmienna a jest mniejsza od 5")
9
```

zmienna a jest mniejsza od 5

```
In [37]: 1 # Zapis warunku w jednej linii
2 a = 15
3 b = 10
4 print("A") if a > b else print("B")
5
```

A

Pętle

For

```
In [38]: 1 # for - pętla wykonuje się przez określoną liczbę iteracji. Nie wymaga
2 # ustawiania zmiennej indeksującej przy
3 # inicjalizacji.
4 liczby = [4, 2, 6, 4, 6]
5
6 for liczba in liczby:
7     # liczba - zmienna w której przechowywany jest każdy kolejny element
```

```

8 # tablicy "liczby". Pętla wykonuje się dopóki nie zostaną wyświetlone
9 # wszystkie elementy z tablicy
10 print(liczba)
11
4
2
6
4
6

```

In [39]:

```

1 # Liczby z wykorzystaniem instrukcji xrange mogą również zawierać się
2 # w określonym przedziale
3 for i in range(5):
4     print(i)
5
0
1
2
3
4

```

In [40]:

```

1 for i in range(5):
2     pass # nie wpływa na działanie programu. Pass stosowane jest, gdy
3         # z różnych powodów pętla lub instrukcja warunkowa nie posiada
4         # zawartości. W przeciwnym wypadku pojawia się błąd.
5

```

In [41]:

```

1 # instrukcja continue odpowiada za opuszczenie instrukcji znajdujących
2 # się poniżej i wraca do wykonywania kodu znajdującego się od początku
3 # pętli. Poniższa instrukcja wypisze tylko liczby parzyste
4 for x in range(30):
5     # Sprawdź, czy x jest nieparzyste
6     if x % 2 != 0:
7         continue
8     print(x)
9
0
2
4
6
8
10
12
14
16
18
20
22
24
26
28

```

While

In [42]:

```

1 # while - pętla wykonuje się dopóki pewien warunek jest spełniony
2 i = 0
3 while i < 5:
4     print(i)
5     i += 1
6
0
1
2
3
4

```

In [43]:

```

1 # instrukcja break używana jest do zakończenia działania pętli for
2 # i while
3 i = 0
4 while True: # pętla nieskończona
5     print(i) # powoduje, że kolejne elementy wyświetlane są dalej
6             # w linii, a nie nowym wierszu

```

```

7     i += 1
8     if i >= 3:
9         break # gdy x >= 3 pętla zostaje przerwana
10
11 print(i)
12
0
1
2
3

```

Funkcje

```

In [44]: 1 # Składnia
2 def nazwa_funkcji():
3     # kod programu
4     pass
5
6 def nazwa_funkcji(argument1, argument2):
7     # można podać więcej argumentów po przecinku
8     # kod programu
9     pass
10
11 def nazwa_funkcji():
12     # kod programu
13     wartosc = 5
14     return wartosc #funkcja zwraca jakąś wartość zapisaną w kodzie programu
15

```

```

In [45]: 1 # Przykładowa funkcja wyświetlająca informację czy dana liczba jest
2 # podzielna przez 3
3 def podzielna_przez_3(liczba):
4     if (liczba % 3) == 0:
5         print("Podana liczba: {:d}, jest podzielna przez 3".format(liczba))
6     else:
7         print("Podana liczba: {:d}, nie jest podzielna przez 3".format(liczba))
8

```

```

In [46]: 1 # Wywołanie funkcji
2 podzielna_przez_3(6)
3 podzielna_przez_3(2)
4

```

Podana liczba: 6, jest podzielna przez 3
Podana liczba: 2, nie jest podzielna przez 3

```

In [47]: 1 def podzielna_przez_3_default_arg(liczba=5): # inicjalizacja argumentu domyślną
2     # wartością
3     if (liczba % 3) == 0:
4         print("Podana liczba: {:d}, jest podzielna przez 3".format(liczba))
5     else:
6         print("Podana liczba: {:d}, nie jest podzielna przez 3".format(liczba))
7

```

```

In [48]: 1 # Wywołanie funkcji
2 # do funkcji z domyślnymi wartościami argumentów nie trzeba przekazywać
3 # ich wartości:
4 podzielna_przez_3_default_arg()
5 # ale można:
6 podzielna_przez_3_default_arg(2)
7

```

Podana liczba: 5, nie jest podzielna przez 3
Podana liczba: 2, nie jest podzielna przez 3

Importowanie modułów

```

In [49]: 1 # importowanie biblioteki
2 import math
3

```

```

In [50]: 1 # Wyświetlenie wszystkich dostępnych funkcji dla modułu math

```

```
2 dir(math)
Out[50]: ['__doc__',
          '__loader__',
          '__name__',
          '__package__',
          '__spec__',
          'acos',
          'acosh',
          'asin',
          'asinh',
          'atan',
          'atan2',
          'atanh',
          'ceil',
          'copysign',
          'cos',
          'cosh',
          'degrees',
          'e',
          'erf',
          'erfc',
          'exp',
          'expm1',
          'fabs',
          'factorial',
          'floor',
          'fmod',
          'frexp',
          'fsum',
          'gamma',
          'gcd',
          'hypot',
          'inf',
          'isclose',
          'isfinite',
          'isinf',
          'isnan',
          'ldexp',
          'lgamma',
          'log',
          'log10',
          'log1p',
          'log2',
          'modf',
          'nan',
          'pi',
          'pow',
          'radians',
          'sin',
          'sinh',
          'sqrt',
          'tan',
          'tanh',
          'trunc']
```

```
In [51]: 1 # Wyświetlanie pomocy do funkcji z biblioteki
          2 help(math.log)
          3
```

Help on built-in function log in module math:

```
log(...)
    log(x[, base])
```

Return the logarithm of x to the given base.
If the base not specified, returns the natural logarithm (base e) of x.

Zmienne globalne

Znajdują się na zewnątrz funkcji lub klasy. Do ich modyfikacji w funkcji lub klasie wykorzystuje się słowo kluczowe **global**.

```
In [52]: 1 x = 10
2
3 def funkcja():
4     global x # wskazuje, że w funkcji będzie modyfikowana zmienna globalna x
5     x = 20
6
7 funkcja()
8
9 print(x)
10
20
```

```
In [53]: 1 x = 10
2
3 def funkcja():
4     # zmienna x widoczna tylko wewnątrz funkcji, nie powoduje zmodyfikowania
5     # globalnej zmiennej x
6     x = 20
7
8 funkcja()
9
10 print(x)
11
10
```

```
In [54]: 1 x = 10
2
3 def funkcja():
4     # użycie zmiennej globalnej x wewnątrz funkcji bez modyfikacji jej
5     # wartości
6     print(x)
7
8 funkcja()
9
10 print(x)
11
10
10
```

Klasy

```
In [55]: 1 class Robot:
2     def __init__(self, name, x, y):
3         # inicjalizacja klasy, wykonuje się zawsze, gdy tworzony jest nowy obiekt
4         # self. - wskazuje na zmienne należące do danej instancji klasy
5         self.name = name
6         self.x = x
7         self.y = y
8
9     def print_pose_info(self): # jedna z metod "print_pose_info" klasy "Robot"
10        print("Pose x={:f}, y={:f}".format(self.x, self.y))
11
12    def move_right(self):
13        self.x += 1
14
15    def move_left(self):
16        self.x -= 1
17
18    def move_up(self):
19        self.y += 1
20
21    def move_down(self):
22        self.y -= 1
23
24    def get_distance_to_start(self):
25        # metoda zwraca odległość robota od startu (0,0)
26        return math.sqrt(self.x*self.x + self.y+self.y)
27
```

```
In [56]: 1 # Inicjalizacja klas Robot
2 robot1 = Robot("r1", 0, 0)
```

```
3 robot2 = Robot("r2", 5, 2)
4
```

```
In [57]: 1 # Odczyt wartości atrybutu klasy
2 robot2.name
3
```

Out[57]: 'r2'

```
In [58]: 1 # Użycie metody klasy
2 robot2.move_right()
3 robot2.print_pose_info()
4
```

Pose x=6.000000, y=2.000000

```
In [59]: 1 # Użycie metody zwracającej jakąś wartość, która została zapisana w zmiennej
2 # dist_to_start
3 dist_to_start= robot2.get_distance_to_start()
4 print("Distance to start: ", dist_to_start)
5
```

Distance to start: 6.324555320336759

```
In [60]: 1 robot1.print_pose_info()
2 robot1.move_right()
3 robot1.print_pose_info()
4
```

Pose x=0.000000, y=0.000000

Pose x=1.000000, y=0.000000

Klasa z Publisherem i Subscriberem

```
In [ ]: 1 import rospy
2 from geometry_msgs.msg import Twist, Pose
3 rospy.init_node('test')
4 # Dla danej klasy w projekcie najlepiej będzie utworzyć nowego node'a
5 # odpowiedzialnego za obsługę interfejsu czy robota.
6
```

```
In [ ]: 1 class Robot:
2     def __init__(self, name, x, y):
3         self.name = name
4         # Dodanie atrybutów z
5         self.velocity_pub = rospy.Publisher('/cmd_vel', Twist, queue_size=10)
6         self.pose_sub = rospy.Subscriber('/pose_topic', Pose, self.callback_pose)
7         self.pose = Pose()
8
9     def callback_pose(self, data):
10        """
11        Metoda wywołuje się gdy na topicu /pose_topic pojawi się dowolna wiadomość.
12        Przerwany zostaje aktualnie wykonywany program i aktualizowana jest wartość
13        atrybutu pose. W pętli sterującej wartość ta może być wykorzystywana do
14        sterowania robotem.
15        """
16        self.pose = data
17
18    def run(self):
19        msg = Twist()
20        self.velocity_pub.publish(msg)
21
```

```
In [ ]: 1 # wysłanie raz prędkości sterującej
2 robot1 = Robot("r1", 0, 0)
3 robot1.run()
4
```

```
In [ ]: 1 # wysyłanie prędkości sterujących w pętli dopóki roscore jest aktywny
2 robot1 = Robot("r1", 0, 0)
3 r = rospy.Rate(10) # 10hz
4
5 not rospy.is_shutdown():
6     # pętla odpowiedzialna za wysyłanie prędkości sterujących robotem. Działa
7     # z częstotliwością 10Hz
8     robot1.run()
```

```
9     r.sleep()
10
```

Klasa z Klientem i Serwerem

```
In [ ]: 1 from pkg_tsr.srv import Task, TaskResponse
2 from turtlesim.srv import Spawn
3
4 class Robot:
5     def __init__(self, name, x, y):
6         self.name = name
7         # utworzenie serwera /go_to_goal
8         self.follow_service = rospy.Service('/go_to_goal', Task, self.go_to_goal)
9         self.create_robot(x,y)
10
11     def create_robot(self, x, y):
12         # Użycie istniejącego klienta do utworzenia robota w symulacji
13         create_new_turtle = rospy.ServiceProxy('spawn', Spawn)
14         create_new_turtle(x, y, 0, self.name)
15
16     def go_to_goal(self, msg_srv):
17         """
18         Tutaj powinien znaleźć się program obsługujący działanie serwera /go_to_goal.
19         Po wysłaniu zapytania przez dowolnego klienta na ten serwer powoduje on, że
20         robot realizuje zadanie dojazdu do punktu. W zależności od tego czy udało się
21         zrealizować dojazd czy nie zwracany jest odpowiedni wynik. Dla kilku robotów
22         nazwy serwerów powinny być związane z nazwami różnych robotów.
23         """
24         # Utworzenie i uzupełnienie odpowiedzi odpowiednimi wartościami.
25         response = TaskResponse()
26
27         # zwrócenie wyniku
28         return response
29
```

Klasa z GUI

```
In [62]: 1 import ipywidgets as widgets
2
3 class GUI:
4     def __init__(self):
5         # Atrybuty z kontrolkami: 2 kontrolki do wprowadzania tekstu i kontrolka
6         # z przyciskiem
7         self.text_a = widgets.Text(value='test', disabled=False,
8                                     description="Text A:")
9         self.text_b = widgets.Text(value='wersja A', disabled=False,
10                                     description="Text B:")
11         self.print_button = widgets.Button(
12             value=False,
13             description='print text',
14             disabled=False,
15             button_style='', # 'success', 'info', 'warning', 'danger'
16             # or ''
17             tooltip='Description',
18             icon='check' # (FontAwesome names without the `fa-` prefix)
19         )
20         # Dodanie obsługi kliknięcia przycisku. Po kliknięciu wywoła się
21         # metoda print_text z tej klasy (GUI)
22         self.print_button.on_click(self.print_text)
23
24     def display_gui(self):
25         # metoda odpowiedzialna za wyświetlenie interfejsu użytkownika
26         # na tym poziomie można ustawić opis z tworzeniem siatki z przyciskami
27         display(self.text_a)
28         display(self.text_b)
29         display(self.print_button)
30
31     def print_text(self, button_data):
32         # metoda wywołująca się po kliknięciu przycisku z napisem print_text
33         print(self.text_a.value + self.text_b.value)
34
```

```
In [63]: 1 # Utworzenie obiektu GUI
2 gui = GUI()
3 # Wyświetlenie interfejsu
4 gui.display_gui()
5
```

Text A:

Text B:

✓ print text

test wersja A

```
In [ ]: 1
```